ITERABLES: LOOP OVER IT

An **iterable** is an object Python can visit one item at a time, usually with a `for` loop.

1) Lists, strings and tuples

A list

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
```

Output :

```
apple
banana
mango
```

A string

```
for letter in "Python":
    print(letter)
```

Output :

```
P
y
t
h
o
n
```

2) Tuples, dictionaries and range

Tuple and dictionary

```
numbers = (10, 20, 30)
for number in numbers:
    print(number)
student = {"name": "Asha", "age": 21}
for key in student:
    print(key, "->", student[key])
```

Output :

```
10
20
30
name -> Asha
age -> 21
```

A range

```
for number in range(1, 5):
    print(number)
```

Output :

```
1
2
3
4
```

looping over a dictionary gives its Keys; range stops before the end value

3) Common iterables

Type	Example	A loop gives
`list`	`[1, 2, 3]`	each item
`str`	`"Python"`	each character
`tuple`	`(10, 20, 30)`	each item
`dict`	`{"a": 1}`	each key
`range`	`range(1, 5)`	1, 2, 3, 4

ITERATORS: ITER() & NEXT()

An **iterator** hands out one item at a time and remembers where it is up to.

1) iter() and next()

Example :

```
numbers = [100, 200, 300]
number_iterator = iter(numbers)
print(next(number_iterator))
print(next(number_iterator))
print(next(number_iterator))
```

Output :

```
100
200
300
```

`iter(list)` get an iterator → `next(it)` ask for one item → no items left: `StopIteration`

2) The end of the line

Example :

```
colors = iter(["red", "green"])
print(next(colors, "No more colors"))
print(next(colors, "No more colors"))
print(next(colors, "No more colors"))
```

Output :

```
red
green
No more colors
```

Example :

```
numbers = iter([1, 2])
next(numbers)
next(numbers)
next(numbers)
```

Output :

```
StopIteration
```

an exhausted iterator does not restart, ask for a fresh one with iter() again

3) What a for loop does for you

Example :

```
names = ["Ali", "Mina", "Ravi"]
names_iterator = iter(names)
while True:
    name = next(names_iterator, None)
    if name is None:
        break
    print(name)
```

Output :

```
Ali
Mina
Ravi
```

a for loop calls iter() once, then next() again and again, and stops quietly on StopIteration

GENERATORS & YIELD

A **generator** is an easy way to build an iterator: `yield` gives a value, then pauses.

1) yield pauses and resumes

Example :

```
def count_up_to_three():
    print("Starting generator...")
    yield 1
    yield 2
    yield 3
    print("Generator finished.")
counter = count_up_to_three()
print(next(counter))
print(next(counter))
print(next(counter))
```

Output :

```
Starting generator...
1
2
3
```

2) Generator functions and expressions

Even numbers

```
def even_numbers(limit):
    number = 2
    while number <= limit:
        yield number
        number += 2
for even in even_numbers(10):
    print(even)
```

Output :

```
2
4
6
8
10
```

Expression and countdown

```
squares = (n * n for n in range(1, 6))
for square in squares:
    print(square)
def countdown(start):
    while start >= 1:
        yield start
        start -= 1
print(list(countdown(3)))
```

Output :

```
1
4
9
16
25
[3, 2, 1]
```

3) Why use a generator?

Example :

```
import sys
as_list = [n for n in range(100000)]
as_generator = (n for n in range(100000))
print("list bytes:", sys.getsizeof(as_list))
print("generator bytes:", sys.getsizeof(as_generator))
```

Output :

```
list bytes: 800984
generator bytes: 192
```

a generator makes values gradually, so it does not build the whole list in memory

Practice & Revision

Write each one yourself first, then compare. Each solution is run for real.

Task 1 Print each of three cities.

Solution :

```
cities = ["Pune", "Mumbai", "Delhi"]
for city in cities:
    print(city)
```

```
Pune
Mumbai
Delhi
```

Task 2 Iterator: first two values.

Solution :

```
scores = [7, 8, 9]
score_iterator = iter(scores)
print(next(score_iterator))
print(next(score_iterator))
```

```
7
8
```

Task 3 Generator countdown(3).

Solution :

```
def countdown(start):
    while start >= 1:
        yield start
        start -= 1
for number in countdown(3):
    print(number)
```

```
3
2
1
```

Task 4 Multiples of five up to 16.

Solution :

```
def multiples_of_five(limit):
    number = 5
    while number <= limit:
        yield number
        number += 5
for number in multiples_of_five(16):
    print(number)
```

```
5
10
15
```

Task 5 Even values by expression.

Solution :

```
values = [1, 2, 3, 4, 5, 6]
even_values = (v for v in values if v % 2 == 0)
for value in even_values:
    print(value)
```

```
2
4
6
```

Task 6 Cubes of 1 to 5.

Solution :

```
cubes = (n ** 3 for n in range(1, 6))
print(list(cubes))
```

```
[1, 8, 27, 64, 125]
```

Quick comparison

Concept	Meaning	Example
Iterable	can be looped over	`[1, 2, 3]`
Iterator	gives the next item, remembers its place	`iter([1, 2, 3])`
Generator	an iterator made with yield	`(x * x for x in range(5))`

every generator is an iterator, but not every iterator is a generator